



Yggdrasil ERP — An Enterprise System Designed to Govern Itself

Mimir Labs Technical Publication

Document:	ML-WP-001	Version:	1.1
Author:	Christopher Gaither	Date:	April 2026
Classification:	Public		

0.1 Why This Matters

The standard enterprise ERP narrative is twenty years old. A vendor sells a flexible platform. A consultancy spends two years configuring it. The customer goes live, discovers the customizations have created an integration island that nothing else speaks to, and pays a permanent tax to maintain it. Five years later, the platform is too entangled to replace, too expensive to keep, and too important to fail. The cycle restarts with a new vendor.

The cost of this cycle is not the license fee. It is the operational opacity that builds up underneath. Audit trails are partial. State transitions are informal. Data ownership is unclear. Reports drift between systems. AI tools cannot be trusted because the data they read does not mean what it says.

Yggdrasil is built on the conviction that none of this is necessary. Governance properties — audit, isolation, lifecycle control, observability, canonical semantics — can be architectural, not bolted-on. An ERP that bakes them in from the first table is qualitatively different from one that adds them as features.

This paper describes what that looks like in practice.

0.2 What Yggdrasil Is

Yggdrasil ERP is a multi-tenant enterprise resource planning platform built for manufacturing and B2B operations. It covers the ten functional modules an industrial enterprise needs — CRM, Sales,



Purchasing, Manufacturing, Warehouse, Finance, Projects, Product Lifecycle Management, Quality, and Service — as one coordinated system rather than a federation of point solutions.

The module set is fixed. There are no customer-defined modules and no schema extensions. That constraint is the point: the platform is a coherent execution environment in which business state is governed by explicit rules, not a configuration kit that can be assembled into something incoherent.

Yggdrasil ships in three forms that share a single canonical data model:

- **A web application** for browser-based access, including self-service portals, onboarding, and a demo environment that provisions an isolated tenant on demand
- **A native desktop client** for the operational environments — manufacturing floors, warehouses, finance departments — where browser interaction is too slow or too thin
- **A backend server** that handles the work, exposes the API, emits the events, and enforces the rules

All three speak the same canonical schema. Whatever you do in one is immediately visible in the others.

0.3 Ten Modules, One System

Customer Relationship Management. Accounts, contacts, opportunities, leads. The starting point for every order-to-cash flow.

Sales. Quotes, orders, invoices, payments, commissions. Every document moves through governed lifecycle stages, not arbitrary status edits.

Purchasing. Purchase orders, suppliers, receipts, bills, three-way matching. Asset registry lives here so capitalization, ownership history, and warranty all flow from acquisition.

Manufacturing. Work orders, bills of materials, shop floor operations, routings, OEE tracking. Sanctioned shop-floor deviations become scoped, temporary derivatives of the released MBOM rather than ad hoc edits to the engineering record.

Warehouse. Inventory, locations, transactions, picking, cycle counts. Real-time visibility, directed putaway and picking.

Finance. General ledger, AR, AP, banking, multi-currency, journals. GL postings carry orthogonal dimension tags — department, project, asset, customer — so the chart of accounts stays stable while reporting can slice along any analytic axis.

Projects. Tasks, time entries, budgets, billing. Engineering work orders, service tickets, and time entries roll up into project cost and revenue.

Product Lifecycle Management. Item masters, engineering and manufacturing BOMs, engineering change requests, revisions. A controlled ECR/ECO/ECN flow connects design intent to produc-



tion reality.

Quality. 8D reports, CAPA actions, NCRs, audits, inspection plans. Inspection failures generate NCRs automatically.

Service. Service tickets, RMAs, warranty, maintenance orders. Field work bills back through projects so cost, parts, and customer invoicing remain linked.

These are not independent applications stitched together. They share a canonical data model, a single state machine, a single audit trail, a single security envelope. Order-to-cash, procure-to-pay, design-to-manufacture, and issue-to-resolution are continuous data flows, not handoffs between disconnected systems.

0.4 What “Governance-Native” Actually Means

Most ERP platforms advertise governance as a feature. In Yggdrasil, governance is the architecture.

Audit by default. Every consequential change — every status transition, every edit to a governed field, every action an automation took on someone’s behalf — lands in a structured audit trail with full deltas, the user who acted, and a timestamp. Nothing has to be wired up; it happens because every write goes through a single audited path.

Tenant isolation enforced by the database. Multi-tenancy is not a query convention. The database itself enforces row-level isolation, so a programming mistake in the application layer cannot leak data between organizations.

Lifecycle states governed by a state machine. Quotes, orders, invoices, work orders, NCRs, ECRs — every governed document has a defined transition graph. Moves outside the graph are rejected. When a valid transition occurs, the status update, the audit entry, and the real-time event are committed together.

Observability built in. Every meaningful change emits a real-time event over the platform’s event hub, available to clients, integration partners, and downstream analytics. There is no polling, no batch reconciliation, no nightly extract. Significant activity is visible the moment it happens.

Canonical semantics. The database schema is the authoritative definition of enterprise meaning. Application code, APIs, and integrations derive from it. External systems adapt to the canonical model, not the other way around. The schema is called Mimisbrunnr; it is the same vocabulary the rest of the Mimir Labs platform uses.

These properties are not optional configurations. A new feature shipped into Yggdrasil inherits them automatically because the platform makes it impossible to do otherwise.



0.5 AI Without Losing Control

Most enterprise software is currently bolting AI onto systems that were never designed to host it. Agents are given write access to live business data with the same trust posture as a human user. When something goes wrong, there is no record of why the agent acted, no proposal that an operator could have rejected, no boundary that prevented the mistake.

Yggdrasil treats agents as first-class participants — but governed by the same architecture that governs human users.

Agents may classify, draft, summarize, or pre-fill, but they cannot independently mutate governed state. Consequential actions arrive as proposals that an operator must explicitly accept, reject, or modify. Every agent action — the tool called, the inputs, the output — lands in the same audit trail that captures human edits. Webhook integrations and external automations route through the same proposal-and-disposition substrate. The result is structural human-in-the-loop, not opt-in.

This applies equally to AI assistants embedded in Yggdrasil itself and to those orchestrated by adjacent tools like Ratatosk's governance chatbot. The platform's posture is consistent: AI participates, but never silently.

0.6 Multi-Tenant by Architecture, Not Convention

Yggdrasil runs many organizations on a single deployment. Each tenant is isolated at every layer:

- The application scopes every request to a tenant context derived from the authenticated session
- The database enforces row-level security so that even a buggy query cannot return another tenant's data
- Real-time event channels and integration messages are scoped to tenant-specific streams
- Configuration, branding, user roles, and audit history are all tenant-local

This makes Yggdrasil suitable for both single-tenant deployments (one organization, dedicated environment) and shared multi-tenant operation (many organizations, one platform, strict separation). The same code path serves both.

0.7 Where Yggdrasil Fits in the Mimir Labs Platform

Yggdrasil is one piece of a larger architecture. The same canonical model that drives the ERP also drives a suite of system-agnostic tools that operate on enterprise data more broadly:

Mimisbrunnr is the canonical model. Yggdrasil's database schema and the universal vocabulary



the rest of the platform speaks. More than three hundred tables across seventeen domains, fixed by design.

Ratatosk discovers what data in your existing systems means. Connects to ERPs, warehouse systems, CRMs, even VBA-driven Excel workbooks. Produces a structured, human-authoritative manifest of enterprise meaning.

Ragnarok migrates data into a target schema using Ratatosk manifests. Deterministic, air-gapped, system-agnostic.

Bifrost maintains live synchronization between enterprise systems once they are aligned. Mirror, unidirectional, or bidirectional sync.

Jormungandr turns governance into an ongoing subscription, continuously validating that enterprise data structures remain compliant.

The tools are independently useful. You can run Ratatosk on your existing SAP environment without ever adopting Yggdrasil. You can use Bifrost to keep two existing systems in sync. The Mimir Labs platform is not a sales funnel that ends in Yggdrasil; it is a coherent set of tools for organizations that take their data architecture seriously, and Yggdrasil happens to be the operational execution layer when one is needed.

0.8 Built for Production

Yggdrasil is engineered for real deployment, not for demos. The server runs as a hardened daemon with health endpoints, structured logging, graceful shutdown, and containerized orchestration. Production deployments sit behind a tunneling layer that handles certificate management and traffic absorption; no application port faces the public internet directly.

The CI pipeline runs lint, type-check, unit tests, schema validation, and full production builds across every component before any deployment. A self-service demo environment exercises the full deployment path on every nightly reset, so the production code path is the same one customers see when they evaluate the platform.

Operational complexity is proportional to business complexity. The system stays small where it can stay small, so the parts that have to be sophisticated can be.

0.9 Why This Matters Again

Enterprise ERP buyers have been promised governance for two decades and have, mostly, received features bolted onto systems that fundamentally do not enforce it. The result is the operational opacity every ERP veteran recognizes: audits that take weeks to assemble, integrations that fail in



non-obvious ways, AI tools that cannot be trusted on the company's own data, migrations that uncover problems no one knew existed.

Yggdrasil is built differently. Tenant isolation is enforced at the database. State transitions go through a single state machine. Audit captures every consequential change. AI participation is gated by the same proposal-and-disposition substrate that gates any consequential change. Canonical semantics make every integration speak the same language at the boundary.

These choices are not neutral. They reflect a particular conviction about what enterprise software is supposed to do: maintain coherent, trustworthy, observable state across operational domains and over time. If that conviction matches the way you want to run your business, Yggdrasil is built for you.

If it does not, there are many other ERPs.

Copyright 2026 Mimir Labs. All rights reserved.

Mimir Labs LLC | Harrisburg, PA

© 2024–2026 Mimir Labs LLC. All rights reserved.

This document contains proprietary information belonging to Mimir Labs LLC. No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of Mimir Labs LLC. The information herein is provided "as is" without warranty of any kind. Product names and trademarks referenced in this document are the property of their respective owners.